

Inventory Management System

A Case Study of Data Structures & Algorithms

Jeremy Roalef, Jesse McDaniel

State University of New York at Morrisville

CITA 225: Introduction to Data Structure

Professor Elfituri

May 1, 2024

Table of Contents

Introduction	4
Text User Interface	4
Functionalities Implemented	4
Add Product	5
Remove Product	5
Update Product Information	6
Add to Cart	7
Remove from Cart	8
Undo Remove from Cart	9
Display Inventory	10
Display Cart	11
Login	12
Sign Up	13
Sign Out	14
Delete Account	15
Get Usernames	16
Set Admin Permission	17
Change Username	18
Change Password	19
Check Account Information	20
Saved Data	21
Data Structures Used	22
Linked List	23
Stack	23
Dictionary	24
List	24
Product	24
User Account	25
Functionalities	26
Colors	27
Graphical User Interface	27

Data Structures Used	28
Lists	28
Sets	28
Tuples	28
Treeview Widget	28
Challenges Faced	28
Jeremy	28
Jesse	29
Lessons Learned	29
Jeremy:	29
Jesse:	29
Figure 2.1.1	5
Figure 2.1.2	6
Figure 2.1.4	7
Figure 2.1.5	7
Figure 2.1.6	8
Figure 2.1.7	9
Figure 2.1.8	10
Figure 2.1.9	11
Figure 2.1.10	11
Figure 2.1.11	12
Figure 2.1.12	13
Figure 2.1.13	13
Figure 2.1.14	14
Figure 2.1.15	14
Figure 2.1.16	15
Figure 2.1.17	15
Figure 2.1.18	16
Figure 2.1.19	16
Figure 2.1.20	17
Figure 2.1.21	18
Figure 2.1.22	18
Figure 2.1.23	19
Figure 2.1.24	20
Figure 2.1.25	21
Figure 2.1.26	21
Figure 2.1.27	22
Figure 2.1.28	22

Introduction

The inventory management system project was designed as a method for companies to create, store, and update product information and allow users to create accounts and purchase products inside the system. This program was created with the usage of 18 functionalities, making use of various data structures, some covered in class and some personally created, to construct a system that is as intuitive as possible. The program works with a text user interface (TUI) and a graphical user interface (GUI). This paper is an in-depth guide to all the functionalities implemented, data structures used, challenges faced, and lessons learned while creating this project.

Text User Interface

The TUI is a complex build of the base project that allows multiple functionalities to occur. The TUI works more personally with the user, permitting them to access certain functionalities that their account type allows. The TUI supports data saving over many program usages, preventing data loss from closing the application. Provided is an in-depth guide to the functionalities and the data structures used in the TUI.

Functionalities Implemented

The inventory management system uses many functionalities for the user accounts. Below is a list of functionalities implemented into the inventory management system:

- Add product
- Remove product
- Update product information
- Add to cart
- Remove from cart
- Undo remove from cart
- Display inventory
- Display cart
- Login
- Sign up
- Sign out
- Delete account
- Get usernames
- Set admin permission
- Change username
- Change password
- Check account information

- Saved data

Add Product

The add product functionality is only accessible to accounts with admin permission. The functionality works by asking the user to enter information about a product, including the product ID, name, price, and quantity. The program will take the information to create an object of type product and store that product in a dictionary and the name in a list (See Figure 2.1.1).

```
Welcome: ADMIN
1. Add Product
2. Remove Product
3. Update Product Information
4. Display Inventory
5. Sign Out
6. Delete Account
7. Get User Names
8. Change User Name
Default: Exit Program
Please enter your Option: 1

Please enter the product ID, or type -1 to exit: 10
Please enter the product Name, or type -1 to exit: Hydraulic Press
Please enter the product price, or type -1 to exit: 299.99
Please enter the quantity of the product, or type -1 to exit: 9
```

Figure 2.1.1

Remove Product

The remove product functionality is only accessible to accounts with admin permission. The functionality works by asking the user to enter a product ID. The program will then loop through the dictionary of products to find and remove the corresponding product ID and return an error message if the product is not found. (See figure 2.1.2 and 2.1.3)

```

Welcome: ADMIN
1. Add Product
2. Remove Product
3. Update Product Information
4. Display Inventory
5. Sign Out
6. Delete Account
7. Get User Names
8. Change User Name
Default: Exit Program
Please enter your Option:
2
Please enter a valid number
Please enter your Option:

Product ID: 1
Product Name: food
Product Price: $8.75
Product Quantity: 8

```

Figure 2.1.2

```

Product ID: 2
Product Name: water
Product Price: $3.99
Product Quantity: 45

Product ID: 3
Product Name: logitech mouse
Product Price: $9.99
Product Quantity: 14

Product ID: 4
Product Name: intro to python by tony gaddis
Product Price: $210.0
Product Quantity: 1

Product ID: 10
Product Name: hydraulic press
Product Price: $299.99
Product Quantity: 9

Please enter the product ID you want to remove, or type -1 to exit: 10

```

Figure 2.1.3

Update Product Information

The update product information functionality is only accessible to accounts with admin permission. The functionality works by asking the user to enter a product ID. The program will look for the product ID in the dictionary of products and then prompt the user to enter what they want to change about the product (name, price, quantity). Finally, the user enters the new information for the respective product information and updates the product information in the dictionary (See figure 2.1.4 and 2.1.5).

```
Welcome: ADMIN
1. Add Product
2. Remove Product
3. Update Product Information
4. Display Inventory
5. Sign Out
6. Delete Account
7. Get User Names
8. Change User Name
Default: Exit Program
Please enter your Option: 3
```

```
Product ID: 1
Product Name: food
Product Price: $8.75
Product Quantity: 8
```

```
Product ID: 2
Product Name: water
Product Price: $3.99
Product Quantity: 45
```

Figure 2.1.3

```
Product ID: 3
Product Name: logitech mouse
Product Price: $9.99
Product Quantity: 14
```

```
Product ID: 4
Product Name: intro to python by tony gaddis
Product Price: $210.0
Product Quantity: 1
```

```
Product ID: 10
Product Name: hydraulic press
Product Price: $299.99
Product Quantity: 9
```

```
Please enter a product ID, or type -1 to exit: 10
```

```
0. Update Product Name
```

```
1. Update Product Price
```

```
2. Update Product Quantity
```

```
What would you like to update for the product: 0
```

```
Please enter the new product Name, or type -1 to exit: hydraulic jack
```

Figure 2.1.4

Add to Cart

The add to cart functionality is accessible to guest and user accounts. The functionality prompts users to enter a number corresponding to a product they can buy. When the user enters a number corresponding to a product, the program will decide if enough of that product exists for the user to add to their cart. If so, the product is added to the user's cart. (See Figure 2.1.6)

```
Welcome: guest
1. Add To Cart
2. Remove From Cart
3. Undo Remove From Cart
4. Display Inventory
5. Display Cart
6. Login
7. Sign Up
8. Check Account Info
Default: Exit Program
Please enter your Option: 1

0. food
1. water
2. logitech mouse
3. intro to python by tony gaddis
4. hydraulic press
please choose an item you want to add to cart, or type -1 to exit: 4
0. food
1. water
2. logitech mouse
3. intro to python by tony gaddis
4. hydraulic press
please choose an item you want to add to cart, or type -1 to exit: 0
adding to empty cart...
```

Figure 2.1.5

Remove from Cart

The remove from cart functionality is accessible to guest and user accounts. The functionality prompts users to enter a number corresponding to a product in their cart. After the user enters a corresponding number, the program will remove one quantity from their cart and loop the functionality until they are satisfied with what they removed. (see figure 2.1.7)


```
Welcome: guest
1. Add To Cart
2. Remove From Cart
3. Undo Remove From Cart
4. Display Inventory
5. Display Cart
6. Login
7. Sign Up
8. Check Account Info
Default: Exit Program
Please enter your Option: 2

0. hydraulic press
What product from cart do you want to remove?, or type -1 to exit: 0
removing...
```

Figure 2.1.6

Undo Remove from Cart

The undo remove from cart functionality is accessible to guest and user accounts. This functionality works by grabbing the user's cart history, removing the top product from it, and adding it back to their cart. (See Figure 2.1.8)

```
Welcome: guest
1. Add To Cart
2. Remove From Cart
3. Undo Remove From Cart
4. Display Inventory
5. Display Cart
6. Login
7. Sign Up
8. Check Account Info
Default: Exit Program
Please enter your Option: 3

Adding back to cart
```

Figure 2.1.7

Display Inventory

The Display inventory functionality is accessible to all account types. This functionality works by iterating through the dictionary of products and displaying the ID, name, price, and quantity of the products to the user. (See figure 2.1.9 and 2.1.10)

```
Welcome: guest
1. Add To Cart
2. Remove From Cart
3. Undo Remove From Cart
4. Display Inventory
5. Display Cart
6. Login
7. Sign Up
8. Check Account Info
Default: Exit Program
Please enter your Option: 4
```

```
Product ID: 1
Product Name: food
Product Price: $8.75
Product Quantity: 7
```

```
Product ID: 2
Product Name: water
Product Price: $3.99
Product Quantity: 45
```

Figure 2.1.8

```
Product ID: 3
Product Name: logitech mouse
Product Price: $9.99
Product Quantity: 14
```

```
Product ID: 4
Product Name: intro to python by tony gaddis
Product Price: $210.0
Product Quantity: 1
```

```
Product ID: 10
Product Name: hydraulic press
Product Price: $299.99
Product Quantity: 8
```

Figure 2.1.9

Display Cart

The display cart functionality is accessible to guest and user accounts. This functionality works by grabbing the current user's shopping cart and printing the cart on the screen. (See Figure 2.1.11)

```
Welcome: guest
1. Add To Cart
2. Remove From Cart
3. Undo Remove From Cart
4. Display Inventory
5. Display Cart
6. Login
7. Sign Up
8. Check Account Info
Default: Exit Program
Please enter your Option: 5

[Product ID: 10
Product Name: hydraulic press
Product Price: $299.99
Product Quantity: 1
]
```

Figure 2.1.10

Login

The login functionality is accessible to guest accounts. The functionality works by prompting the user to enter a username and password. If the username and password pair do not match any entry in the user account dictionary, the user is prompted with an incorrect username or password error and asked if they would like to try again. If an entry is found in the user account dictionary, the current user is changed to the correct user account. Notice that the program will now welcome the new user's name instead of the guest's (See Figure 2.1.12 and 2.1.13)

```

Welcome: guest
1. Add To Cart
2. Remove From Cart
3. Undo Remove From Cart
4. Display Inventory
5. Display Cart
6. Login
7. Sign Up
8. Check Account Info
Default: Exit Program
Please enter your Option: 6

Username: helloworld!
Password: password
Incorrect username or password
Try Again (enter positive number for yes, negative number for no)? 1
Username: sampleUser
Password: 111

```

Figure 2.1.11

```

Welcome: sampleUser
1. Add To Cart
2. Remove From Cart
3. Undo Remove From Cart
4. Display Inventory
5. Display Cart
6. Sign Out
7. Delete Account
8. Set Admin Permission
9. Change User Name
10. Change Password
11. Check Account Info
12. Proceed To Checkout
13. Add Funds
Default: Exit Program
Please enter your Option:

```

Figure 2.1.12

Sign Up

The sign up functionality is accessible to guest accounts. The functionality prompts the user to enter a username and a password. Afterward, a new user account object is created and added to the user account dictionary. Notice that the program now welcomes the new username instead of the guest. (See figure 2.1.14 and 2.1.15)

```
Welcome: guest
1. Add To Cart
2. Remove From Cart
3. Undo Remove From Cart
4. Display Inventory
5. Display Cart
6. Login
7. Sign Up
8. Check Account Info
Default: Exit Program
Please enter your Option: 7
```

```
Username: superCoolUsername!
Password: superSecurePassword!
Re-enter Password: superSecurePassword!
```

Figure 2.1.13

```
Welcome: superCoolUsername!
1. Add To Cart
2. Remove From Cart
3. Undo Remove From Cart
4. Display Inventory
5. Display Cart
6. Sign Out
7. Delete Account
8. Set Admin Permission
9. Change User Name
10. Change Password
11. Check Account Info
12. Proceed To Checkout
13. Add Funds
Default: Exit Program
Please enter your Option:
```

Figure 2.1.14

Sign Out

The sign out functionality is accessible to guest and admin accounts. The functionality works by taking the current account that is signed in and assigning it to the premade guest account. Notice that the program now welcomes guests. (See figure 2.1.16 and 2.1.17)

```
Welcome: superCoolUsername!  
1. Add To Cart  
2. Remove From Cart  
3. Undo Remove From Cart  
4. Display Inventory  
5. Display Cart  
6. Sign Out  
7. Delete Account  
8. Set Admin Permission  
9. Change User Name  
10. Change Password  
11. Check Account Info  
12. Proceed To Checkout  
13. Add Funds  
Default: Exit Program  
Please enter your Option: 6
```

Figure 2.1.15

```
Welcome: guest  
1. Add To Cart  
2. Remove From Cart  
3. Undo Remove From Cart  
4. Display Inventory  
5. Display Cart  
6. Login  
7. Sign Up  
8. Check Account Info  
Default: Exit Program  
Please enter your Option:
```

Figure 2.1.16

Delete Account

The delete account functionality is accessible to user and admin accounts. The functionality prompts users to enter “DELETE” to delete their account or “no” to return to the main page. When the user enters “DELETE,” the program will remove the account from the user account dictionary and then assign the current user back to the predetermined guest account. Notice that the program welcomes guests after account deletion. (See figure 2.1.18 and 2.1.19)

```
Welcome: superCoolUsername!  
1. Add To Cart  
2. Remove From Cart  
3. Undo Remove From Cart  
4. Display Inventory  
5. Display Cart  
6. Sign Out  
7. Delete Account  
8. Set Admin Permission  
9. Change User Name  
10. Change Password  
11. Check Account Info  
12. Proceed To Checkout  
13. Add Funds  
Default: Exit Program  
Please enter your Option: 7  
  
Are you sure you want to delete your account?  
Type 'DELETE' to delete your account, or type 'no' to exit  
DELETE
```

Figure 2.1.17

```
Welcome: guest  
1. Add To Cart  
2. Remove From Cart  
3. Undo Remove From Cart  
4. Display Inventory  
5. Display Cart  
6. Login  
7. Sign Up  
8. Check Account Info  
Default: Exit Program  
Please enter your Option:
```

Figure 2.1.18

Get Usernames

The get usernames functionality is accessible to admin accounts. The functionality works by iterating through each entry in the user account dictionary and printing the account's username. (See Figure 2.1.20)


```
Welcome: ADMIN
1. Add Product
2. Remove Product
3. Update Product Information
4. Display Inventory
5. Sign Out
6. Delete Account
7. Get User Names
8. Change User Name
Default: Exit Program
Please enter your Option: 7

jeremy
sampleUser
ADMIN
testdummy1
```

Figure 2.1.19

Set Admin Permission

The set admin permission functionality is accessible to user accounts. The functionality works by prompting the user to enter the admin password. If they do, the account's permission attribute will change from "user" to "admin," allowing the account to use and only use admin permission. Notice that the functionalities displayed to the user change after setting admin permission. (See figure 2.1.21 and 2.1.22)

```
Welcome: testdummy1
1. Add To Cart
2. Remove From Cart
3. Undo Remove From Cart
4. Display Inventory
5. Display Cart
6. Sign Out
7. Delete Account
8. Set Admin Permission
9. Change User Name
10. Change Password
11. Check Account Info
12. Proceed To Checkout
13. Add Funds
Default: Exit Program
Please enter your Option: 8

Please enter the password, or type 'no' to exit: CITA225
```

Figure 2.1.20

```
Welcome: testdummy1
1. Add Product
2. Remove Product
3. Update Product Information
4. Display Inventory
5. Sign Out
6. Delete Account
7. Get User Names
8. Change User Name
Default: Exit Program
Please enter your Option:
```

Figure 2.1.21

Change Username

The change username functionality is accessible to user and admin accounts. The functionality works by prompting the user to enter a new username. Afterward, the account's username attribute will be set to the new username. Notice that the program now welcomes the new username after running the functionality. (See Figure 2.1.23)

```
Welcome: testdummy1
1. Add Product
2. Remove Product
3. Update Product Information
4. Display Inventory
5. Sign Out
6. Delete Account
7. Get User Names
8. Change User Name
Default: Exit Program
Please enter your Option: 8

New Username: notADummy1

Welcome: notADummy1
1. Add Product
2. Remove Product
3. Update Product Information
4. Display Inventory
5. Sign Out
6. Delete Account
7. Get User Names
8. Change User Name
Default: Exit Program
Please enter your Option:
```

Figure 2.1.22

Change Password

The change password functionality is accessible to user accounts. The functionality works by prompting the user to enter their current password followed by their new password and a re-enter new password. Afterward, the program will take the user's password attribute and set it to the new password. (See Figure 2.1.24)

```
Welcome: jeremy
1. Add To Cart
2. Remove From Cart
3. Undo Remove From Cart
4. Display Inventory
5. Display Cart
6. Sign Out
7. Delete Account
8. Set Admin Permission
9. Change User Name
10. Change Password
11. Check Account Info
12. Proceed To Checkout
13. Add Funds
Default: Exit Program
Please enter your Option: 10

Type 'no' at any moment to return to main menu.
Please enter your password: 123
New Password: password
Re-enter Password: password
```

Figure 2.1.23

Check Account Information

The check account information functionality is accessible to guest and user accounts. The functionality works by taking the current user's account and printing their information. (See figure 2.1.25 and 2.1.26)

```
Welcome: jeremy
1. Add To Cart
2. Remove From Cart
3. Undo Remove From Cart
4. Display Inventory
5. Display Cart
6. Sign Out
7. Delete Account
8. Set Admin Permission
9. Change User Name
10. Change Password
11. Check Account Info
12. Proceed To Checkout
13. Add Funds
Default: Exit Program
Please enter your Option: 11
```

Figure 2.1.24

```
UserName: jeremy
Password: password
Balance: 0
Shopping Cart:
[Product ID: 10
Product Name: hydraulic press
Product Price: $299.99
Product Quantity: 1
]
Cart History:

Account Type: user
```

Figure 2.1.25

Saved Data

The saved data functionality is inaccessible to any account but works behind the scenes. The functionality creates a new Python file called “savedData.py” with write permissions. The program will then iterate through the product dictionary, the product list, and the user accounts, and each attribute information will be written over to the new file. Each data structure is stored in a list or a 2-D list, as objects cannot be transferred over through file writing. When the user reruns the program, the program initializes each data structure by reading the lists of information in the savedData.py file, appending all products created with their respective attributes, all product names, and all user account information. (See figure 2.1.27 and 2.1.28)

```

Welcome: jeremy
1. Add To Cart
2. Remove From Cart
3. Undo Remove From Cart
4. Display Inventory
5. Display Cart
6. Sign Out
7. Delete Account
8. Set Admin Permission
9. Change User Name
10. Change Password
11. Check Account Info
12. Proceed To Checkout
13. Add Funds
Default: Exit Program
Please enter your Option: -1

Process finished with exit code 0

```

Figure 2.1.26

```

dictionaryInformation = [[1, 'food', 8.75, 7], [2, 'water', 3.99, 45], [3, 'logitech mouse', 9.99, 14], [4, 'intro to python by tony gaddis', 29.99, 1], [5, 'hydraulic press', 299.99, 1]]
listInformation = ['food', 'water', 'logitech mouse', 'intro to python by tony gaddis', 'hydraulic press']
userAccountInformation = [['jeremy', 'password', 0, 'user', [[10, 'hydraulic press', 299.99, 1]], []], ['sampleUser', '111', 0, 'user', [

```

Figure 2.1.27

Data Structures Used

This program uses various data structures, some covered in class and some personally created. Below is a list of all the data structures used in this project:

- Linked List
- Stack
- Dictionary
- List
- Product
- User Account
- Functionalities
- Colors

Linked List

The linked list class is a data structure that behaves similarly to a typical list with the benefits of faster insertion and removal. The linked list class works by storing elements, called nodes, in a sequence such that the first element, called the head, points to the second element, the second points to the third, and so on until the final node, called the tail, that points to nothing. Only the head and tail contents are immediately accessible to the user; every other node will have to be found by starting at the head element, looking at the element it points to, and comparing that element to the element we want to insert. See Figure 2.2.1 for a UML diagram showing the functionalities of the class.

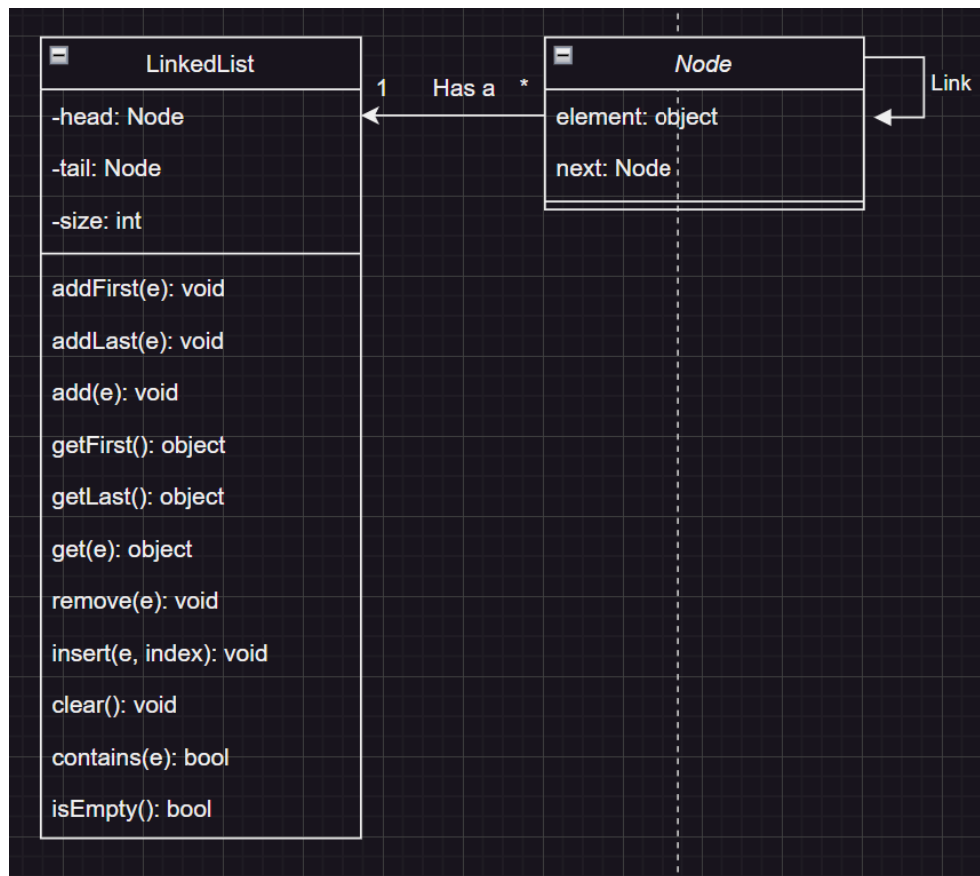


Figure 2.2.1

Stack

The stack class is a data structure where you can only send and retrieve elements. It works if the first element in the stack is the last one that comes out (FILO) or the last element in the stack is the first one out (LIFO). See Figure 2.2.2 for a UML diagram showing the methods and attributes of the stack.

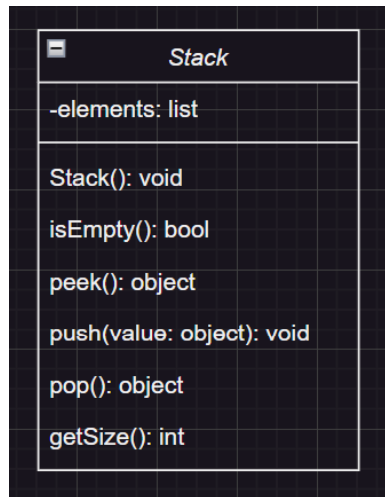


Figure 2.2.2

Dictionary

The dictionary class is a built-in Python data structure that works similarly to a list. The difference is that a dictionary is well-equipped to find values based on a given key rather than an index. In other words, a dictionary works by taking any value for a key and using it to find an entry in the dictionary. The dictionary is not limited to using numbers for searching, making it much easier to find elements than a list would.

List

The list is a built-in Python data structure. The data structure stores elements of similar or different types in an indexed sequence. The index values start at 0 and increments by 1 for each sequence. You can use the index of an element to find and locate values in the list. The list class could be faster and easier to use but is a basis for other practical classes like stacks and queues used in many projects.

Product

The product class is a data structure that was personally created. The product class is a storage system for unique products in an inventory system. The class's functionalities are only used to get and set product values. Everything else relating to products is determined in other classes or the program. See Figure 2.2.3 for a UML diagram of the product class.

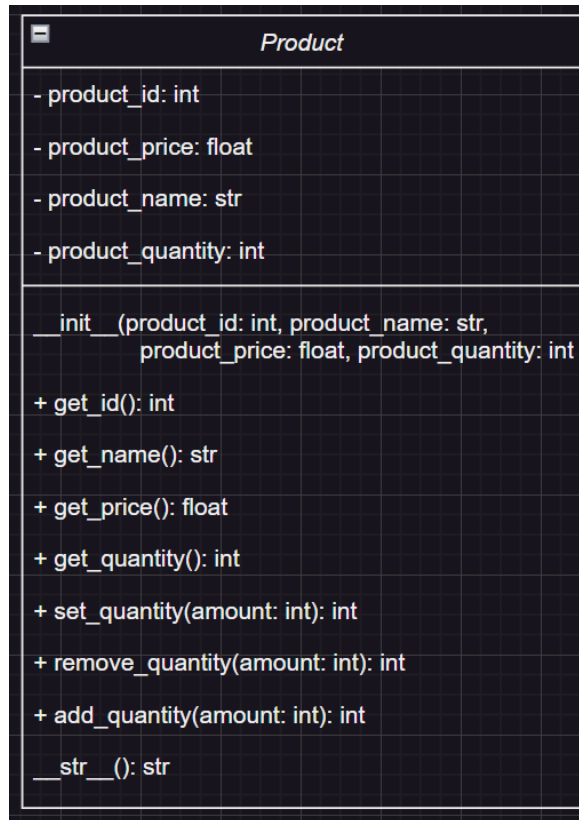


Figure 2.2.3

User Account

The user account class is a data structure personally created. This class stores many other data structures, including the linked lists and stacks. User accounts were made with the idea of users with different permissions being able to use the program in various ways. For example, guest accounts can create a new account or log in to an existing one, but they cannot add or remove products from the inventory management system. The user account class has many methods and attributes, as shown in Figure 2.2.4.

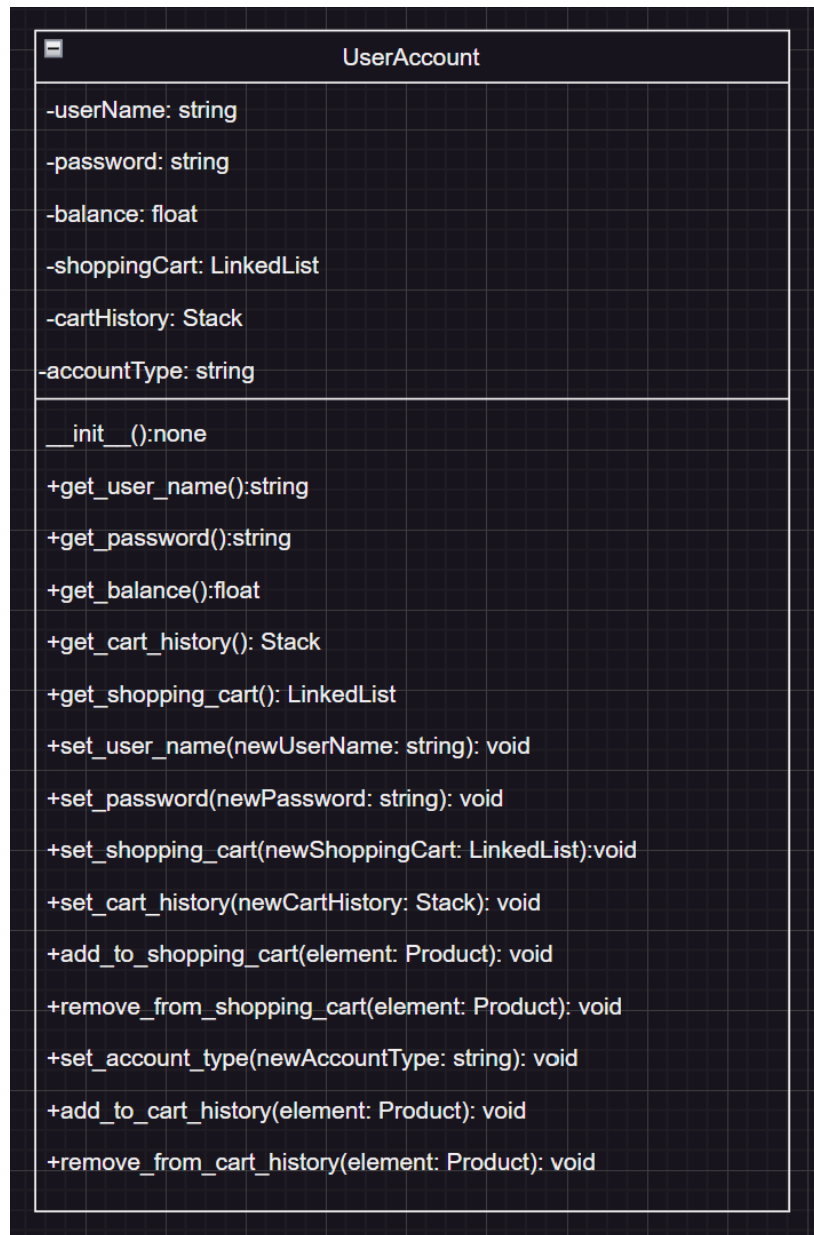


Figure 2.2.4

Functionalities

The functionalities class is a data structure designed to help add new functionalities to the program. The class works by assigning the functionality with a name, a text, and a permissions attribute intended to call the function in code, display the function text to the user, and only display it to account types that match the functionality's permissions. See Figure 2.2.5 for a UML diagram of the Functionalities class.

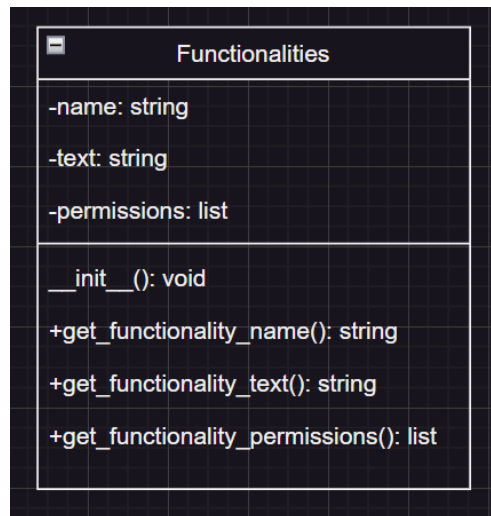


Figure 2.2.5

Colors

The colors class is a data structure with various American National Standards Institute (ANSI) escape code values to add color to the program. This class works by changing the current ANSI escape code value to a new one corresponding to a color. For example, blue has an ANSI escape code value of '\033[34m'. See Figure 2.2.6 for a UML diagram of the Colors class.

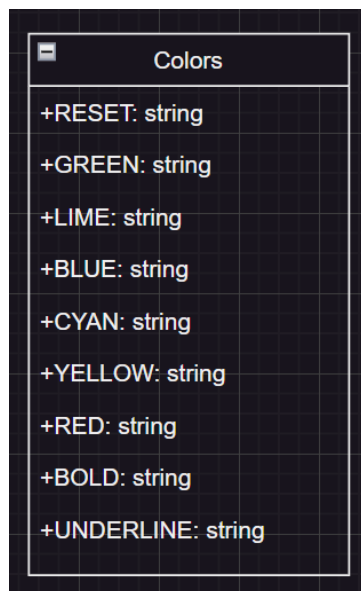


Figure 2.2.6

Graphical User Interface

I (Jesse) made a GUI version of the project that meets all the requirements. I Used Tkinter, which lets you build GUI applications with Python, to make this happen. It utilizes tkinter widgets like Toplevel, Label, Entry, OptionMenu, Button, and Treeview to create windows, display text and

input fields, and handle user interactions. The application structure includes classes for adding, removing, and updating products in the inventory, showing the inventory and cart, and managing cart functionalities. The App class serves as the main driver for the application, initializing the GUI and handling user actions through button clicks. Tkinter is very similar to Visual Studio except that it doesn't have a GUI, so everything, such as buttons and textboxes, must be entered manually. Each Window that opens up is in a different class.

Data Structures Used

Lists

Lists store data such as dictionary Information, product_ids, product_names, and product_options.

Sets

Sets are used to store unique product IDs in unique_product_ids, displayed_product_ids, and deleted_product_ids

Tuples

Tuples are used to represent individual products in the dictionaryInformation list. Each tuple contains information about a product, such as its ID, name, price, and quantity.

Treeview Widget

Tkinter provides the Treeview widget, which is used to display data in the display inventory and display cart visually.

Challenges Faced

Jeremy

The base project was a breeze. The real challenge came when Jesse and I wanted to expand on the project. For me, that was adding new features and functionalities to the Main.py file. The greatest challenge was the application of user accounts into the base project. By this point in time, I had taken a 2-week break from working on the project, and I had to reevaluate what we made and figure out how I could add user accounts. The problem with adding new features and functionalities to an ever-expanding project is that it becomes harder to determine where the new feature must interact with the project. For example, while adding the user account class, I noticed it no longer made sense to display certain functionalities to the user. So, I designed a new method of displaying functionalities (through the functionalities class) to display functionalities better so the user can use them with their account.

Another challenge I faced came from outside the project: using GitHub. Jesse and I used GitHub to share updates on the project. One of the problems we ran into was when we both worked on

the project simultaneously without the other knowing, and I accidentally overwrote Jesse's commit. Fortunately, the work did not mix, so it was a simple fix but still troublesome.

Jesse

One challenge that I faced was getting Tkinter to work. The application was often brand new to me, and I had to watch and follow many tutorials online to figure out how to get everything to work. Another challenge I faced with this project was all the exception errors that occurred and trying to fix those, which was resolved using many error messages. Another issue I encountered was getting the saved data functionality to work correctly, which I eventually did by watching many tutorials. Other challenging things I faced were testing and debugging the project over minor issues and trying to figure out where the problem was in the code and fix it.

Lessons Learned

Jeremy:

I learned more about what it is like to work on a project with a team member as passionate as I was on the project. I learned not only how to design the TUI better but also how to design the code interface. I tried to make the code read itself with no magic numbers and comments provided when needed. I learned how to use online resources like ChatGPT to quickly gather ideas on approaching a particular task, using the technology as a base for further research. For example, I used ChatGPT to learn how to color the TUI using ANSI escape code values.

I learned how to prioritize tasks in a project with a fixed deadline. This project's challenge was not completing the base project functionalities but instead completing extensions on the base project. There are still features missing that Jesse and I wanted to add, but given the time constraint, we never got around to making them happen. One of these features was hooking up the GUI with the TUI, allowing both systems to work as one instead of separately. In the end, we prioritized the user experience over another functionality.

Jesse:

I have learned by doing this project how to use Tkinter to create windows, labels, buttons, entry fields, dropdown menus, and Treeview widgets for displaying visual data. I have also learned how to make executables for Python applications, though I found it only works for applications with a GUI. I've learned how to organize Python GUI elements using frames and grid layouts to arrange buttons and widgets in a structured manner. I've learned about functionality to handle user input, such as adding new products, updating quantities, removing products, and adding products to the cart. I've also learned how to save and load data using the pickle module, allowing the application to store and retrieve inventory and cart data even after closing the

program. I've also learned much about using classes and objects to represent products and windows.